



Pontificia Universidad
JAVERIANA
Colombia

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Sistemas Distribuidos

Guía de Implementación de HDFS en una Red LAN con Ubuntu y Rocky Linux

Integrantes:

Daniel Ramírez
Guillermo Aponte
Samuel Pico
Ana Sofía Arboleda
Paula Losada

Docente: John Corredor Franco

Fecha: 8 de Mayo de 2026

Índice

1. Pruebas de éxito en el laboratorio	3
2. Sistema de ficheros distribuido con Hadoop	5
2.1. Introducción	5
2.2. Propósito del documento	5
2.3. Alcance	5
2.4. Qué es HDFS	5
2.5. Diagrama de funcionamiento de HDFS	6
2.6. Diagrama de funcionamiento de YARN	6
2.7. Utilidades	7
2.8. Heterogeneidad de nuestro cluster	7
2.9. Compatibilidad entre Ubuntu y Rocky Linux para la creación de nuestro cluster	8
2.10. Separación de roles entre NameNode y DataNode	9
2.11. ¿Puede el NameNode funcionar también como DataNode?	10
2.12. Ventajas y riesgos de mezclar roles en una misma máquina	11
2.13. Relación entre HDFS, YARN y MapReduce	12
2.14. Distribución de almacenamiento frente a distribución de procesamiento	13
2.15. Tolerancia a fallos y replicación de bloques	13
2.16. Escalabilidad del cluster	15
2.16.1. Cómo escalar el cluster una vez ya esté hecho	17
3. Guía completa para crear HDFS en una red LAN con Ubuntu y Rocky Linux	22
3.1. Versión, alcance y seguridad	22
3.2. Diseño del cluster en LAN	23
3.2.1. Topología utilizada	23
3.2.2. Puertos importantes	23
3.3. Preparación de las máquinas	23
3.3.1. Paso 1. Fijar hostnames	23
3.3.2. Paso 2. Configurar resolución de nombres	24
3.3.3. Paso 3. Instalar dependencias	24
3.3.4. Paso 4. Crear usuario hadup2	25
3.4. Instalación de Hadoop	26
3.4.1. Paso 5. Descargar Hadoop en el NameNode	26
3.4.2. Paso 6. Instalar Hadoop en /opt	26
3.4.3. Paso 7. Copiar Hadoop a los DataNodes	27
3.5. SSH sin contraseña entre nodos	27
3.5.1. Paso 8. Generar llave SSH en el NameNode	27
3.6. Configuración de Hadoop	28
3.6.1. Paso 9. Crear directorios locales de HDFS	28
3.6.2. Paso 10. Configurar hadoop-env.sh	28
3.6.3. Paso 11. Configurar core-site.xml	29
3.6.4. Paso 12. Configurar hdfs-site.xml	29
3.6.5. Paso 13. Configurar workers	30
3.6.6. Paso 14. Configurar YARN y MapReduce	30

3.6.7. Paso 15. Distribuir la configuración	32
3.7. Arranque del cluster	32
3.7.1. Paso 16. Formatear HDFS	32
3.7.2. Paso 17. Iniciar HDFS	32
3.7.3. Paso 18. Iniciar YARN	33
3.8. Pruebas de HDFS	33
3.8.1. Paso 19. Crear directorio de usuario en HDFS	33
3.8.2. Paso 20. Subir un archivo a HDFS	33
3.8.3. Paso 21. Descargar y comparar	34
3.8.4. Paso 22. Verificar bloques y réplicas	34
3.8.5. Paso 23. Probar tolerancia básica a fallos	34
3.9. Prueba opcional con MapReduce	35
3.10. Apagado del cluster	35
3.11. Problemas encontrados durante la práctica	36
3.11.1. El DataNode no aparece en la web del NameNode	36
3.11.2. SSH sigue pidiendo contraseña	36
3.11.3. Error de JAVA_HOME	36
4. Conclusiones	37
5. Referencias	37

Resumen

Este documento se organiza en dos partes. La primera reserva el espacio conceptual para explicar el sistema de ficheros distribuido construido con Hadoop. La segunda presenta la guía completa para instalar, configurar, arrancar y probar Apache Hadoop HDFS sobre máquinas Linux conectadas en una red LAN, usando Ubuntu en la mayoría de equipos y Rocky Linux en el DataNode de Ana Sofía.

1. Pruebas de éxito en el laboratorio

Las siguientes capturas registran que el cluster quedó funcionando correctamente durante la prueba en la red LAN del laboratorio.

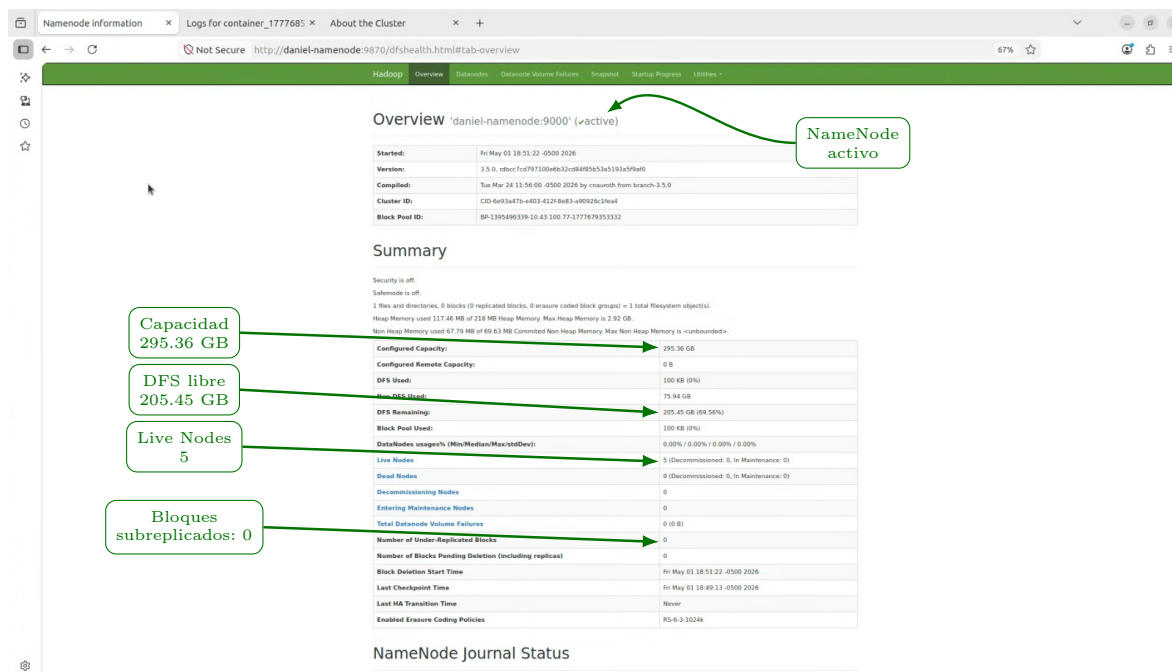


Figura 1: Prueba de HDFS: interfaz del NameNode mostrando el estado general del sistema de archivos distribuido.

- El NameNode aparece activo.
- El clúster reporta 5 Live Nodes y 0 Dead Nodes.
- La capacidad configurada es de 295.36 GB y el espacio DFS disponible es de 205.45 GB.
- El sistema reporta 0 bloques subreplicados.
- Estos valores evidencian que los nodos estaban conectados y que HDFS reconocía correctamente el almacenamiento distribuido.

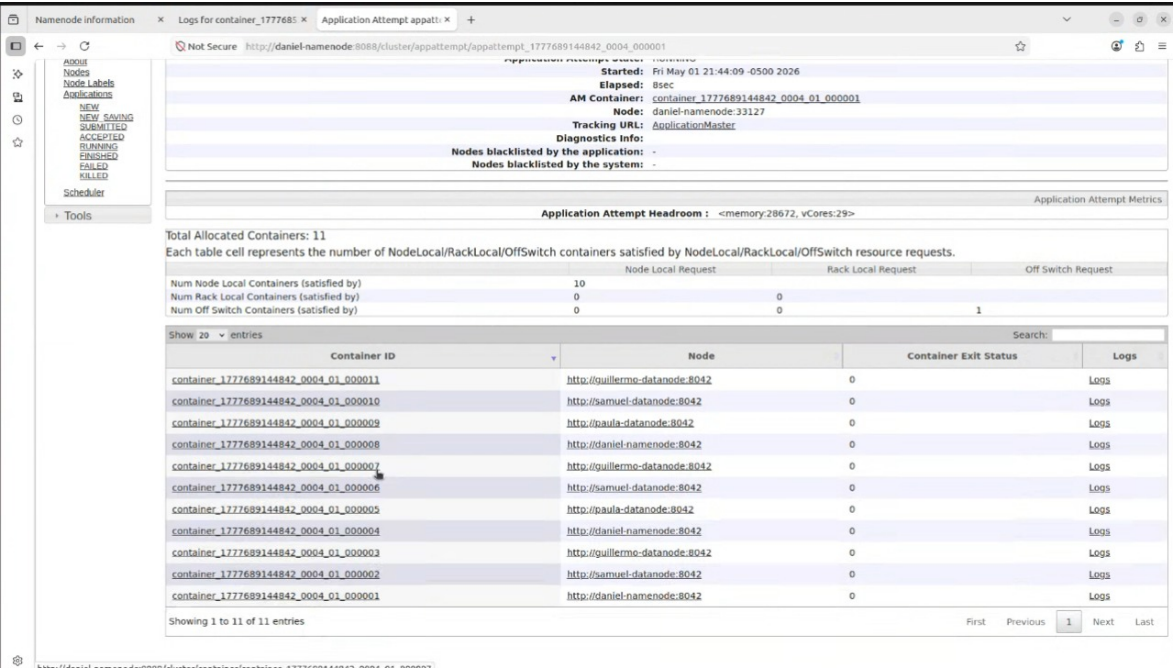


Figura 2: Prueba con YARN: distribución de contenedores y cargas de procesamiento entre los nodos del cluster.

2. Sistema de ficheros distribuido con Hadoop

2.1. Introducción

Un sistema de ficheros distribuido (SFD) permite almacenar y gestionar información de manera conjunta entre múltiples máquinas conectadas en red, de tal forma que el usuario percibe un único sistema lógico de almacenamiento. Este enfoque es fundamental cuando los volúmenes de datos superan la capacidad de un solo equipo o cuando se requiere alta disponibilidad y tolerancia a fallos.

En este contexto, Apache Hadoop ofrece una solución robusta mediante su sistema de archivos distribuido HDFS (Hadoop Distributed File System), el cual permite dividir archivos en bloques y distribuirlos entre varios nodos del cluster. Esto facilita no solo el almacenamiento eficiente, sino también el acceso paralelo a los datos.

En una red LAN, como la utilizada en este laboratorio, HDFS permite simular arquitecturas de almacenamiento distribuido similares a las empleadas en entornos empresariales o de Big Data, aprovechando múltiples equipos físicos para construir un sistema escalable, tolerante a fallos y de alto rendimiento.

2.2. Propósito del documento

El propósito de este documento es servir como guía teórica y práctica para la implementación de un sistema de ficheros distribuido utilizando Apache Hadoop HDFS en un entorno de red local. Se busca que el lector comprenda tanto los conceptos fundamentales como los pasos técnicos necesarios para desplegar un cluster funcional.

Adicionalmente, el documento pretende demostrar el funcionamiento real de HDFS mediante pruebas en laboratorio, evidenciando la correcta comunicación entre nodos, la distribución de datos y la tolerancia a fallos. También se incluye la integración con YARN para ampliar el alcance hacia el procesamiento distribuido.

2.3. Alcance

Este documento cubre la instalación, configuración y validación de un cluster Hadoop en una red LAN utilizando sistemas operativos Ubuntu y Rocky Linux. Se enfoca en la implementación de HDFS como sistema de almacenamiento distribuido y en la configuración básica de YARN para ejecución de tareas.

El alcance está limitado a un entorno de laboratorio controlado, sin configuraciones avanzadas de seguridad como, cifrado de datos o control de accesos. Tampoco se abordan escenarios de producción, balanceo avanzado de carga ni optimizaciones de alto nivel.

El sistema construido permite almacenar, replicar y recuperar archivos distribuidos entre los nodos del cluster, así como ejecutar pruebas básicas de procesamiento distribuido. Sin embargo, no se consideran aspectos como alta disponibilidad del NameNode ni despliegues en la nube.

2.4. Qué es HDFS

HDFS (Hadoop Distributed File System) es un sistema de archivos distribuido diseñado para almacenar grandes volúmenes de datos de manera confiable en clusters

de hardware convencional. Su arquitectura se basa en la división de archivos en bloques de tamaño fijo, los cuales se distribuyen y replican en diferentes nodos del sistema.

El funcionamiento de HDFS se apoya en dos tipos principales de componentes:

- **NameNode:** es el nodo maestro encargado de gestionar los metadatos del sistema, como la ubicación de los bloques, la estructura de directorios y los permisos de acceso.
- **DataNodes:** son los nodos trabajadores que almacenan físicamente los bloques de datos y sus réplicas.

Cuando un cliente accede a un archivo, primero consulta al NameNode para conocer la ubicación de los bloques y luego se comunica directamente con los DataNodes para leer o escribir la información. Este diseño permite un alto rendimiento y escalabilidad.

Además, HDFS implementa replicación de bloques (por defecto tres copias), lo que garantiza tolerancia a fallos. Si un DataNode falla, los datos pueden recuperarse desde otras réplicas disponibles en el cluster.

2.5. Diagrama de funcionamiento de HDFS

El siguiente diagrama resume la interacción principal entre el cliente, el NameNode y los DataNodes dentro de HDFS.

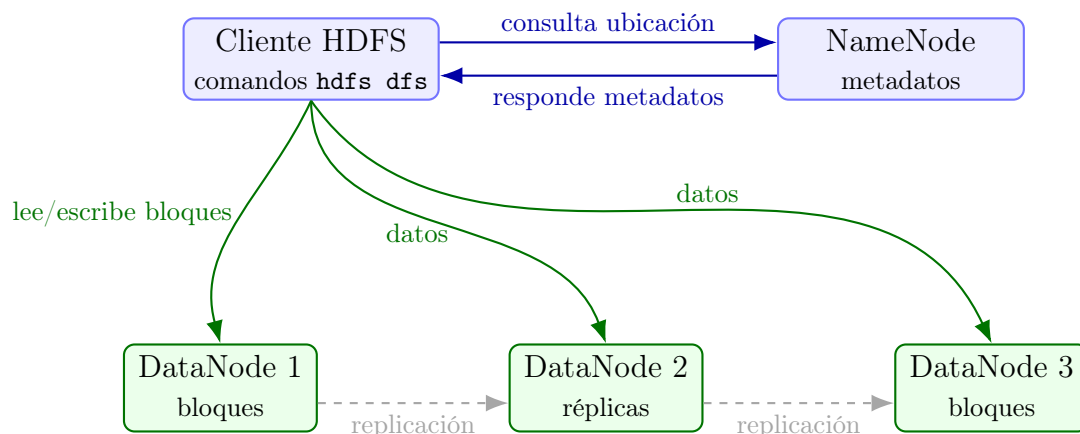


Figura 3: Funcionamiento general de HDFS: el NameNode administra metadatos y los DataNodes almacenan bloques y réplicas.

2.6. Diagrama de funcionamiento de YARN

El siguiente diagrama muestra cómo YARN coordina los recursos del cluster para ejecutar tareas distribuidas sobre los nodos disponibles.

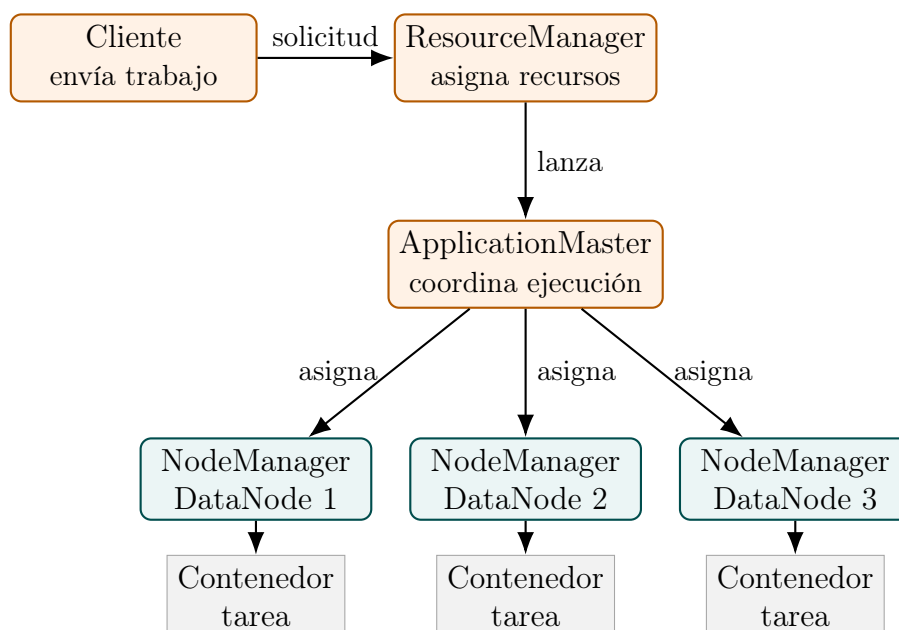


Figura 4: Funcionamiento general de YARN: el ResourceManager administra recursos y los NodeManagers ejecutan contenedores de trabajo.

2.7. Utilidades

El uso de HDFS resulta especialmente útil en escenarios donde se manejan grandes volúmenes de datos o se requiere alta disponibilidad del almacenamiento. Algunas de sus principales aplicaciones incluyen:

- **Almacenamiento de Big Data:** permite gestionar grandes conjuntos de datos que no pueden ser almacenados en un solo equipo.
- **Procesamiento distribuido:** en conjunto con YARN y MapReduce, facilita la ejecución de tareas paralelas sobre los datos almacenados.
- **Tolerancia a fallos:** gracias a la replicación de bloques, el sistema puede seguir funcionando incluso si uno o varios nodos fallan.
- **Escalabilidad:** es posible agregar nuevos nodos al cluster para aumentar la capacidad de almacenamiento sin interrumpir el sistema.
- **Uso en entornos académicos y de laboratorio:** permite simular arquitecturas reales de sistemas distribuidos utilizando recursos limitados.

En comparación con el almacenamiento local tradicional, HDFS ofrece ventajas significativas en términos de disponibilidad, distribución de carga y capacidad de crecimiento, lo que lo convierte en una herramienta clave en sistemas modernos de análisis de datos.

2.8. Heterogeneidad de nuestro cluster

Nuestro cluster es heterogéneo porque está compuesto por máquinas que no tienen exactamente las mismas características de software. Aunque todos los nodos hacen

parte de la misma arquitectura Hadoop, no todos utilizan la misma distribución de Linux: la mayoría de equipos trabajan con Ubuntu y uno de los DataNodes utiliza Rocky Linux. Esta combinación convierte el ambiente de laboratorio en un escenario mixto, más cercano a lo que puede ocurrir en sistemas distribuidos reales.

La heterogeneidad no se limita únicamente al sistema operativo. También se refleja en la forma en que cada máquina administra paquetes, servicios, permisos, rutas de instalación y variables de entorno. Por ejemplo, en Ubuntu se usa el gestor de paquetes `apt`, mientras que en Rocky Linux se utiliza `dnf`. De igual manera, algunos servicios pueden tener nombres distintos, como `ssh` en Ubuntu y `sshd` en Rocky Linux. Estas diferencias obligan a adaptar ciertos comandos durante la instalación y configuración del cluster.

A pesar de estas diferencias, Hadoop permite trabajar sobre un conjunto de nodos heterogéneos siempre que se mantenga una configuración lógica común. En nuestro caso, fue necesario estandarizar elementos como el usuario de ejecución `hadup2`, los nombres de host, el archivo `/etc/hosts`, la comunicación por SSH, la versión de Java y los archivos de configuración de Hadoop. Gracias a esta estandarización, los nodos pudieron comunicarse correctamente dentro de la red LAN y cumplir sus respectivos roles dentro de HDFS y YARN.

Esta heterogeneidad también permitió evidenciar una característica importante de los sistemas distribuidos: los nodos no necesitan ser idénticos para trabajar en conjunto, pero sí deben ser compatibles y estar correctamente configurados. El NameNode puede administrar metadatos, los DataNodes pueden almacenar bloques y los NodeManagers pueden ejecutar tareas siempre que todos reconozcan la misma configuración del cluster.

En conclusión, la heterogeneidad de nuestro cluster representó un reto adicional durante la práctica, pero también enriqueció el proceso de implementación. Al combinar Ubuntu y Rocky Linux, fue necesario identificar diferencias entre sistemas operativos, ajustar comandos específicos y verificar que todos los nodos cumplieran los requisitos necesarios para integrarse a la infraestructura Hadoop.

2.9. Compatibilidad entre Ubuntu y Rocky Linux para la creación de nuestro cluster

La compatibilidad entre Ubuntu y Rocky Linux dentro de nuestro cluster fue posible porque ambos sistemas pertenecen al ecosistema Linux y pueden ejecutar los componentes necesarios para Hadoop. Aunque son distribuciones diferentes, comparten características fundamentales como soporte para Java, servicios de red, usuarios del sistema, permisos sobre archivos y ejecución de procesos en segundo plano.

Para Hadoop, lo más importante no es que todos los nodos tengan exactamente la misma distribución, sino que todos puedan ejecutar los mismos servicios bajo una configuración común. En nuestro caso, esto significó que cada nodo debía reconocer el mismo NameNode, comunicarse correctamente dentro de la red LAN, ejecutar los procesos correspondientes de HDFS y YARN, y mantener una estructura coherente de usuarios, rutas y permisos.

La compatibilidad también dependió de mantener una versión uniforme de Java en todos los nodos. Como Hadoop se ejecuta sobre la máquina virtual de Java, esta capa funciona como un punto común entre Ubuntu y Rocky Linux. Gracias a esto, los procesos de Hadoop pueden ejecutarse de forma similar en ambas distribuciones, aunque internamente cada sistema administre sus paquetes y servicios de manera distinta.

Otro aspecto clave fue la configuración compartida de Hadoop. Los archivos como `core-site.xml`, `hdfs-site.xml`, `yarn-site.xml`, `mapred-site.xml` y `workers` permitieron que todos los nodos interpretaran la misma arquitectura del cluster. Esto hizo posible que los DataNodes se conectaran al NameNode y que los NodeManagers se integraran al ResourceManager sin importar si el sistema operativo era Ubuntu o Rocky Linux.

Ubuntu y Rocky Linux fueron compatibles para la creación del cluster porque ambos cumplían los requisitos esenciales de Hadoop: ejecución de Java, conectividad de red, comunicación remota, manejo de usuarios y configuración uniforme. Las diferencias entre distribuciones no afectaron el funcionamiento general, siempre que se mantuvieran consistentes los elementos críticos del entorno distribuido.

2.10. Separación de roles entre NameNode y DataNode

En HDFS la separación de roles es fundamental para organizar el funcionamiento del sistema de archivos distribuido. Cada componente cumple una responsabilidad distinta: el NameNode administra la estructura lógica del sistema, mientras que los DataNodes almacenan físicamente los bloques de datos.

Aspecto	NameNode	DataNode
Rol principal	Actúa como nodo maestro del sistema HDFS.	Actúa como nodo trabajador encargado del almacenamiento.
Función	Administra los metadatos del sistema de archivos.	Guarda los bloques reales de los archivos.
Tipo de información que maneja	Estructura de directorios, nombres de archivos, permisos, ubicación de bloques y estado de los nodos.	Bloques de datos y réplicas almacenadas localmente.
Relación con el cliente	El cliente lo consulta para saber dónde están los archivos o bloques.	El cliente se comunica con ellos para leer o escribir los datos.
Fallo del componente	Si falla, el sistema pierde temporalmente la capacidad de ubicar y administrar archivos.	Si falla uno, HDFS puede seguir funcionando si existen réplicas en otros DataNodes.
Escalabilidad	Normalmente no se agregan muchos NameNodes en una configuración básica.	Se pueden agregar más DataNodes para aumentar la capacidad del cluster.
Ejemplo de proceso esperado	NameNode y, en configuraciones básicas, SecondaryNameNode.	DataNode.

Cuadro 1: Comparación entre el rol del NameNode y los DataNodes en HDFS.

En nuestro cluster, esta división permitió definir una arquitectura clara: un nodo se encarga de la administración de metadatos y los demás nodos se encargan del almacenamiento distribuido. Esto ayudó a verificar con mayor facilidad qué servicios

debían estar activos en cada máquina y a comprobar el funcionamiento correcto de HDFS durante la práctica.

2.11. ¿Puede el NameNode funcionar también como DataNode?

Sí, el *NameNode* puede funcionar también como *DataNode*. Hadoop permite que una misma máquina ejecute varios procesos al mismo tiempo, por lo que técnicamente es posible que un nodo actúe como nodo maestro y, además, almacene bloques de datos. Esta configuración suele usarse en ambientes de prueba, laboratorios académicos o instalaciones pequeñas, donde no siempre se cuenta con muchas máquinas disponibles.

Sin embargo, aunque sea posible, no siempre es lo más recomendable. En HDFS, el *NameNode* cumple una función central, porque administra los metadatos del sistema de archivos distribuido. Esto significa que no guarda directamente los archivos de los usuarios, sino la información necesaria para saber dónde están ubicados los bloques, qué *DataNodes* los contienen, cómo está organizada la estructura de directorios, cuáles son los permisos y cuál es el estado general del sistema.

Por otro lado, los *DataNodes* sí se encargan del almacenamiento físico de los bloques. Estos nodos guardan los datos reales y reportan periódicamente su estado al *NameNode*. Por esta razón, cuando se mezclan los dos roles en una misma máquina, ese equipo debe encargarse tanto de la administración lógica del sistema como del almacenamiento físico de información.

Aspecto	NameNode separado del DataNode	NameNode también como DataNode
Organización del cluster	Los roles quedan claramente separados: un nodo administra y otros almacenan.	La misma máquina administra metadatos y almacena bloques.
Uso de recursos	El NameNode concentra sus recursos en la gestión de metadatos.	La máquina consume recursos en dos tareas: administración y almacenamiento.
Facilidad para detectar errores	Es más fácil identificar si el problema está en el nodo maestro o en un nodo trabajador.	Puede ser más difícil saber si el error viene del rol de NameNode o del rol de DataNode.
Uso recomendado	Más adecuado para clusters distribuidos y prácticas con varias máquinas.	Adecuado para pruebas pequeñas, ambientes de aprendizaje o configuraciones de un solo nodo.
Riesgo ante fallos	Si falla un DataNode, HDFS puede seguir funcionando si existen réplicas.	Si falla esa máquina, se afecta el nodo maestro y también una parte del almacenamiento.

Cuadro 2: Comparación entre mantener separado el NameNode y mezclarlo con el rol de DataNode.

En un laboratorio pequeño esto puede funcionar sin problema, porque la cantidad de datos y la carga del sistema son limitadas. No obstante, en un escenario más grande, esta mezcla de roles puede generar mayor consumo de CPU, memoria, disco y red en el nodo maestro. Además, si esa máquina falla, se afecta tanto la administración del sistema como una parte del almacenamiento.

En nuestro cluster se decidió mantener una separación clara de roles. El equipo de Daniel Ramírez actúa como *NameNode*, *ResourceManager* y cliente principal del sistema, mientras que los equipos de Guillermo Aponte, Samuel Pico, Ana Sofía Arboleda y Paula Losada funcionan como *DataNodes* y *NodeManagers*. Esta separación permite entender mejor la arquitectura de Hadoop, porque diferencia el nodo encargado de coordinar el sistema de los nodos encargados de almacenar datos y ejecutar tareas.

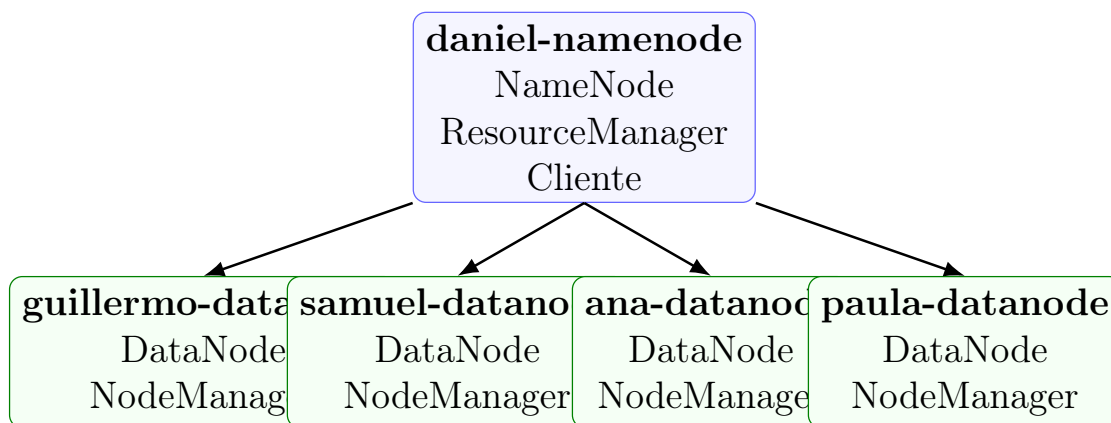


Figura 5: Separación de roles usada en el cluster del laboratorio.

También facilita la revisión de errores. Si un *DataNode* no aparece en la interfaz web del *NameNode*, se puede revisar específicamente la configuración de ese nodo trabajador, su conexión por red, su entrada en el archivo `workers`, los permisos o los logs del *DataNode*. En cambio, si todos los roles estuvieran mezclados en la misma máquina, sería más difícil identificar si el problema corresponde a almacenamiento, metadatos, red o configuración general.

En conclusión, el *NameNode* sí puede funcionar como *DataNode*, pero en este proyecto se mantiene separado para representar de forma más clara una arquitectura distribuida. Esta decisión hace que el cluster sea más ordenado, más fácil de administrar y más cercano al funcionamiento real de HDFS.

2.12. Ventajas y riesgos de mezclar roles en una misma máquina

Mezclar roles en una misma máquina es una decisión común en clusters pequeños o de laboratorio, porque reduce la cantidad de equipos necesarios y simplifica la puesta en marcha. Una sola computadora puede asumir funciones de administración, coordinación e incluso servir como punto principal desde el cual se ejecutan los comandos del cliente. Esta estrategia resulta útil cuando el objetivo principal es aprender la arquitectura, validar conectividad y comprobar que el sistema funciona de extremo a extremo.

Entre las ventajas más claras se encuentran la facilidad de administración, el menor consumo de infraestructura y la reducción del trabajo de configuración. Al concentrar

varios servicios en un solo equipo, es más sencillo revisar procesos, consultar logs y controlar el arranque o apagado del cluster. En un entorno académico, esto también permite aprovechar mejor los recursos disponibles sin exigir una máquina dedicada para cada componente.

Sin embargo, mezclar roles también introduce riesgos importantes. Cuando una misma máquina concentra demasiadas responsabilidades, compite por memoria, CPU y disco con varios procesos de Hadoop al mismo tiempo. Esto puede afectar el rendimiento general y volver más difícil el diagnóstico de fallos, porque un problema local puede impactar simultáneamente la administración del sistema, la coordinación de tareas o el acceso del cliente.

Desde la perspectiva de diseño distribuido, esta práctica es aceptable en pruebas controladas, pero deja de ser conveniente cuando el cluster crece o cuando se busca mayor estabilidad. Por eso, en implementaciones más exigentes se intenta separar los servicios críticos para disminuir la sobrecarga y reducir puntos únicos de fallo. En nuestro contexto de laboratorio, combinar algunos roles administrativos fue una solución práctica, pero mantener el almacenamiento repartido entre varios DataNodes siguió siendo fundamental.

2.13. Relación entre HDFS, YARN y MapReduce

HDFS, YARN y MapReduce forman tres partes complementarias del ecosistema Hadoop, pero cada una cumple una función distinta. HDFS se encarga del almacenamiento distribuido, YARN administra los recursos de cómputo del cluster y MapReduce ofrece un modelo de ejecución para procesar datos en paralelo. En conjunto, estos componentes permiten no solo guardar información en varias máquinas, sino también trabajar sobre ella de manera coordinada.

La relación entre ellos comienza cuando los archivos se almacenan en HDFS. Una vez los datos están divididos en bloques y distribuidos entre los DataNodes, YARN puede organizar el uso de CPU y memoria de los nodos para ejecutar tareas sobre esa información. De esta manera, el sistema no se limita a conservar los datos, sino que también estructura la forma en que serán procesados dentro del cluster.

MapReduce aparece en esta arquitectura como un mecanismo de procesamiento distribuido. Un trabajo se divide en tareas que pueden ejecutarse en diferentes nodos, mientras YARN decide dónde lanzar los contenedores y supervisa su ejecución. Al finalizar, los resultados pueden almacenarse de nuevo en HDFS, de modo que el flujo completo integra almacenamiento, administración de recursos y procesamiento.

En nuestra práctica, esta relación fue importante porque HDFS permitió validar el sistema de ficheros distribuido y YARN amplió el alcance de la implementación hacia la ejecución de cargas distribuidas. Esto demuestra que Hadoop no es solamente una herramienta para guardar archivos, sino una plataforma compuesta por varios servicios que cooperan entre sí. Aunque HDFS, YARN y MapReduce pueden estudiarse por separado, su valor real aparece cuando operan de forma articulada dentro del mismo cluster.

2.14. Distribución de almacenamiento frente a distribución de procesamiento

En un sistema distribuido no es lo mismo repartir el almacenamiento que repartir el procesamiento. La distribución de almacenamiento consiste en fragmentar los archivos en bloques y ubicarlos en varios nodos, de manera que la información quede repartida y replicada dentro del cluster. La distribución de procesamiento, en cambio, consiste en dividir una carga de trabajo en tareas que se ejecutan en diferentes máquinas para aprovechar el paralelismo.

En Hadoop, HDFS resuelve principalmente la primera necesidad. Su propósito es aumentar la capacidad disponible, mantener copias de los bloques y permitir que los datos continúen accesibles incluso si algún nodo falla. YARN, junto con motores como MapReduce, atiende la segunda necesidad, porque organiza dónde se ejecuta cada tarea y cómo se usan los recursos de cómputo del cluster.

Aunque ambos tipos de distribución son diferentes, están estrechamente relacionados. Cuando el procesamiento se ejecuta cerca de los nodos donde están almacenados los bloques, disminuye el tráfico innecesario en la red y mejora la eficiencia general del sistema. Por esta razón, Hadoop busca coordinar almacenamiento y ejecución, en lugar de tratarlos como procesos aislados.

En nuestro laboratorio, la distribución de almacenamiento se evidenció en la capacidad agregada de HDFS y en la presencia de varios DataNodes activos, mientras que la distribución de procesamiento se observó en YARN mediante el reparto de contenedores y cargas entre nodos. Esta distinción ayuda a entender que un cluster no solo debe guardar datos en múltiples máquinas, sino también ser capaz de aprovecharlas para procesar esa información. En consecuencia, almacenamiento distribuido y procesamiento distribuido son conceptos complementarios, pero no equivalentes.

2.15. Tolerancia a fallos y replicación de bloques

La tolerancia a fallos es una de las características más importantes de HDFS. En un sistema distribuido no se puede asumir que todas las máquinas estarán funcionando siempre de forma correcta. Un nodo puede apagarse, perder conexión con la red, quedarse sin recursos, reiniciarse o presentar errores temporales. Por eso, HDFS está diseñado para que la información no dependa de una única máquina.

Para lograr esto, HDFS divide los archivos en bloques y guarda varias copias de cada bloque en diferentes DataNodes. Estas copias se conocen como réplicas. De esta manera, si un DataNode falla, el sistema puede seguir accediendo a los datos desde otra réplica disponible en otro nodo del cluster.

En la configuración de nuestro cluster se definió la propiedad `dfs.replication` con valor 3. Esto significa que HDFS intenta mantener tres copias de cada bloque almacenado. Como el cluster cuenta con cuatro DataNodes, Hadoop puede distribuir esas tres réplicas entre diferentes equipos.

Por ejemplo, un mismo bloque puede quedar replicado de la siguiente forma:

- una copia en `guillermo-datanode`,
- otra copia en `samuel-datanode`,
- y una tercera copia en `ana-datanode`.

Si uno de esos nodos deja de estar disponible, todavía existirán otras copias desde las cuales se puede leer la información.

También se configuró el tamaño de bloque mediante la propiedad `dfs.blocksize`, con valor `134217728`, que corresponde a 128 MB. Esto quiere decir que los archivos almacenados en HDFS se dividen en bloques de ese tamaño. Si el archivo es pequeño, puede ocupar un solo bloque; si es más grande, se divide en varios bloques. Cada bloque se replica según el factor de replicación configurado.

Elemento	Descripción en el cluster
<code>dfs.replication=3</code>	Cada bloque debe tener tres copias distribuidas entre los DataNodes disponibles.
<code>dfs.blocksize=134217728</code>	Cada bloque tiene un tamaño de 128 MB. Los archivos grandes se dividen en varios bloques.
Cantidad de DataNodes	El cluster cuenta con cuatro DataNodes: Guillermo, Samuel, Ana Sofía y Paula.
Ventaja principal	Si un DataNode falla, el archivo puede seguir leyéndose desde otra réplica disponible.
Comando de verificación	<code>hdfsfsck/user/hadup2/prueba.txt-files-blocks-locations</code>
Reporte general	<code>hdfsdfsadmin-report</code>

Cuadro 3: Configuración usada para la replicación y verificación de bloques en HDFS.

El funcionamiento general es el siguiente: cuando un cliente quiere leer un archivo, primero consulta al NameNode. El NameNode responde indicando en qué DataNodes se encuentran los bloques del archivo. Después, el cliente se comunica directamente con los DataNodes para leer los datos. Si una réplica no está disponible porque un DataNode falló, el cliente puede usar otra réplica del mismo bloque.

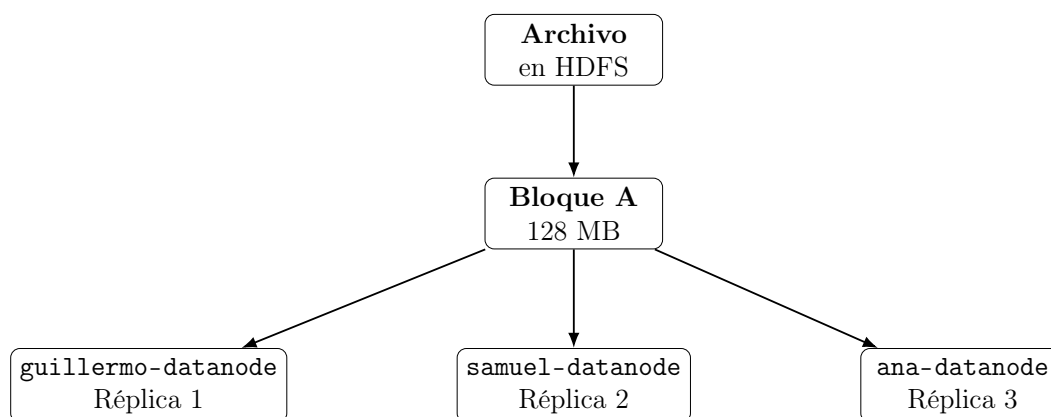


Figura 6: Ejemplo de replicación de un bloque con factor de replicación 3.

Esta replicación permite que HDFS mantenga la disponibilidad de los datos incluso si ocurre un fallo parcial. En nuestro laboratorio, la tolerancia básica a fallos se prueba deteniendo temporalmente un DataNode mediante SSH y luego intentando leer un archivo desde HDFS. Si el archivo puede leerse correctamente, significa que las réplicas restantes están funcionando y que el sistema no dependía únicamente del nodo detenido.

Para revisar el estado de los bloques y sus réplicas se utiliza el comando:

Paso 1: Inspeccionar archivo en HDFS

```
hdfs fsck /user/hadup2/prueba.txt -files -blocks -locations
```

Este comando permite ver los bloques del archivo, la cantidad de réplicas y las ubicaciones donde se encuentran almacenados. Con esto se puede comprobar si HDFS está distribuyendo correctamente los datos entre los DataNodes.

Además, para revisar el estado general del sistema se puede usar:

Paso 2: Revisar estado general de HDFS

```
hdfs dfsadmin -report
```

Este comando muestra información sobre los DataNodes activos, el espacio disponible, el espacio usado y el estado general del sistema de archivos distribuido. En la interfaz web del NameNode también se puede revisar si los nodos están vivos, si hay nodos caídos y si existen bloques subreplicados.

Situación	Qué ocurre en HDFS	Resultado esperado
Todos los DataNodes activos	Cada bloque puede mantener sus tres réplicas.	El sistema aparece saludable y sin bloques subreplicados.
Un DataNode se detiene temporalmente	HDFS intenta leer los datos desde las réplicas restantes.	El archivo debería seguir siendo accesible si existen otras copias.
Quedan menos réplicas de las configuradas	El bloque puede aparecer como subreplicado.	HDFS intentará recuperar el nivel de replicación cuando haya nodos disponibles.
El DataNode vuelve a activarse	El nodo se reporta nuevamente al NameNode.	El cluster puede recuperar estabilidad después de unos segundos.

Cuadro 4: Comportamiento esperado de HDFS ante fallos parciales.

Un bloque subreplicado aparece cuando HDFS no tiene la cantidad de copias esperada según el valor de `dfs.replication`. Por ejemplo, si el factor de replicación es 3 y solo quedan 2 copias disponibles de un bloque, HDFS lo puede reportar como subreplicado. En ese caso, cuando haya nodos disponibles, el sistema puede intentar crear nuevas réplicas para recuperar el nivel de protección configurado.

En nuestro caso, la replicación de bloques es fundamental porque permite demostrar que HDFS no funciona como un simple almacenamiento local, sino como un sistema distribuido. Los archivos no quedan guardados en una sola máquina, sino repartidos y replicados entre varios DataNodes. Esto permite que el sistema sea más confiable frente a fallos individuales.

2.16. Escalabilidad del cluster

La escalabilidad del cluster se refiere a la capacidad que tiene el sistema para crecer cuando aumentan las necesidades de almacenamiento o procesamiento. En HDFS, esta

escalabilidad se logra principalmente agregando nuevos DataNodes al cluster. Cada nuevo DataNode aporta más espacio en disco y permite distribuir mejor los bloques de datos.

Esta forma de crecimiento se conoce como escalabilidad horizontal. En lugar de depender de una sola máquina cada vez más potente, el sistema puede crecer incorporando más equipos conectados a la red. Esta característica es importante en los sistemas distribuidos, porque permite aprovechar varios computadores para construir una infraestructura más grande.

Tipo de crecimiento	Descripción	Aplicación en HDFS
Escalabilidad vertical	Aumentar los recursos de una sola máquina, como más RAM, más CPU o más disco.	Puede ayudar, pero no es la forma principal de crecimiento de HDFS.
Escalabilidad horizontal	Agregar más máquinas al sistema.	Es el enfoque más natural en HDFS, porque se agregan más DataNodes al cluster.
Escalabilidad en almacenamiento	Aumentar la capacidad disponible para guardar archivos.	Se logra agregando DataNodes con más espacio de disco.
Escalabilidad en procesamiento	Aumentar la capacidad para ejecutar tareas distribuidas.	Se logra agregando NodeManagers para que YARN tenga más recursos.

Cuadro 5: Tipos de escalabilidad relacionados con el cluster Hadoop.

En nuestro cluster, la arquitectura ya está diseñada de forma distribuida: un equipo cumple el rol de *NameNode* y los demás cumplen el rol de *DataNode*. Por eso, si en el futuro se necesitara aumentar la capacidad, se podría agregar una nueva máquina a la red LAN, instalarle las dependencias necesarias, configurar Hadoop y registrarla como nuevo nodo trabajador.

Al agregar un nuevo DataNode, HDFS puede empezar a usarlo para almacenar nuevos bloques. Esto aumenta la capacidad total del sistema y también puede ayudar a repartir mejor la carga de lectura y escritura. Mientras más DataNodes tenga el cluster, más opciones tiene HDFS para distribuir los datos y sus réplicas.

La escalabilidad también se relaciona con YARN. HDFS se encarga del almacenamiento distribuido, mientras que YARN administra los recursos de procesamiento. Por eso, si el nuevo nodo también se configura como *NodeManager*, el cluster no solo gana más almacenamiento, sino también más capacidad para ejecutar tareas distribuidas.

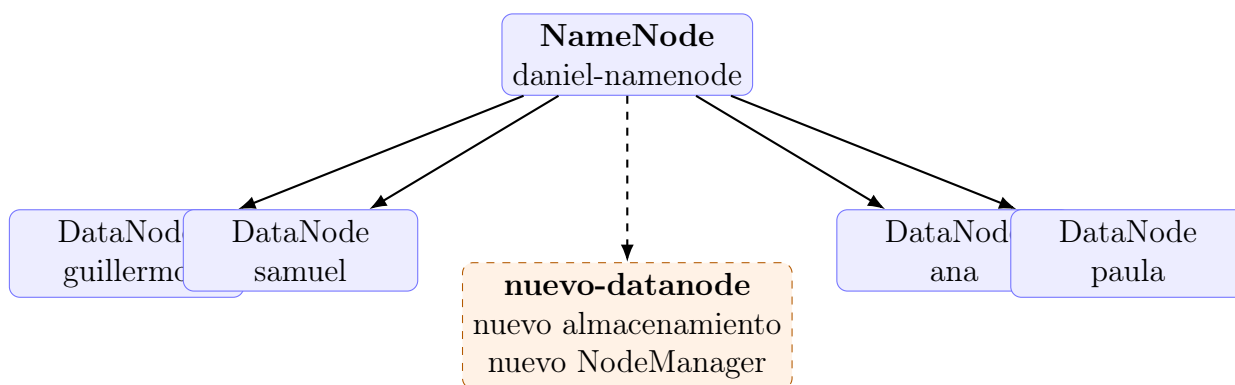


Figura 7: Escalabilidad horizontal del cluster al agregar un nuevo DataNode.

Sin embargo, escalar el cluster no consiste únicamente en conectar otra máquina. Para que un nuevo nodo funcione correctamente, debe cumplir las mismas condiciones básicas del resto del sistema. Debe estar en la misma red LAN, resolver correctamente los nombres de host, tener Java instalado, contar con Hadoop en la misma ruta, usar el mismo usuario de ejecución, tener comunicación SSH con el NameNode y recibir los mismos archivos de configuración.

También es importante actualizar el archivo `workers` en el NameNode, porque este archivo indica qué máquinas deben ser tratadas como nodos trabajadores. Si el nuevo nodo no aparece allí, los scripts de Hadoop no lo tendrán en cuenta al iniciar los servicios de HDFS o YARN.

En conclusión, el cluster es escalable porque permite agregar nuevos DataNodes para ampliar la capacidad de almacenamiento y, si se configura YARN, también la capacidad de procesamiento. Esta característica permite que HDFS se adapte al crecimiento del sistema sin tener que rediseñar toda la arquitectura desde cero.

2.16.1. Cómo escalar el cluster una vez ya esté hecho

Para escalar el cluster una vez ya está construido, se debe agregar una nueva máquina como DataNode. Si también se desea que participe en el procesamiento distribuido, se debe configurar además como NodeManager. A continuación se presenta el procedimiento completo, manteniendo la misma lógica usada en la guía del laboratorio.

Paso	Acción	Objetivo
1	Conectar la nueva máquina a la LAN	Garantizar que el nuevo nodo esté en la misma red que el cluster.
2	Asignar hostname	Identificar el nodo con un nombre claro, por ejemplo <code>nuevo-datanode</code> .
3	Actualizar <code>/etc/hosts</code>	Permitir que todos los nodos resuelvan el nombre del nuevo equipo.
4	Instalar dependencias	Tener Java 17, SSH, rsync, curl y tar instalados.
5	Crear usuario <code>hadup2</code>	Mantener el mismo usuario de ejecución en todo el cluster.

Paso	Acción	Objetivo
6	Copiar Hadoop desde el NameNode	Usar la misma versión y estructura de instalación.
7	Configurar variables de entorno	Permitir que el sistema reconozca <code>JAVA_HOME</code> , <code>HADOOP_HOME</code> y rutas de Hadoop.
8	Crear directorio local del DataNode	Definir dónde se guardarán los bloques de HDFS en la nueva máquina.
9	Configurar SSH sin contraseña	Permitir que el NameNode pueda iniciar servicios remotos.
10	Agregar el nodo a <code>workers</code>	Registrar el nuevo equipo como trabajador del cluster.
11	Copiar configuración Hadoop	Asegurar que el nuevo nodo tenga los mismos archivos de configuración.
12	Iniciar DataNode y NodeManager	Activar el almacenamiento y, si aplica, el procesamiento distribuido.
13	Verificar con <code>dfsadmin</code> y web UI	Confirmar que el nuevo nodo aparece activo en HDFS.

Cuadro 6: Resumen del proceso para escalar el cluster agregando un nuevo DataNode.

Primero, la nueva máquina debe conectarse a la misma red LAN del cluster. Después se le debe asignar un nombre de host claro, por ejemplo:

Paso 3: Asignar hostname al nuevo nodo

```
sudo hostnamectl set-hostname nuevo-datanode
```

Luego, en todas las máquinas del cluster se debe actualizar el archivo `/etc/hosts` para que todos los nodos puedan resolver el nombre del nuevo equipo. Se agrega una línea con la IP real de la máquina y su hostname:

Paso 4: Agregar el nuevo nodo a `/etc/hosts`

```
10.43.xxx.xxx nuevo-datanode
```

Después se verifica la conectividad desde el NameNode:

Paso 5: Probar conectividad con el nuevo nodo

```
ping -c 3 nuevo-datanode
```

Si el ping falla, primero se debe corregir la IP, la conexión de red, el firewall, el hostname o la configuración del archivo `/etc/hosts`. Hadoop no funcionará correctamente si los nodos no pueden comunicarse por nombre.

Luego se instalan las dependencias necesarias. Si el nuevo nodo usa Ubuntu, se ejecuta:

Paso 6: Dependencias en Ubuntu para el nuevo nodo

```
sudo apt update
sudo apt install -y openjdk-17-jdk openssh-server rsync curl tar
sudo systemctl enable --now ssh
java -version
```

Si el nuevo nodo usa Rocky Linux, se ejecuta:

Paso 7: Dependencias en Rocky Linux para el nuevo nodo

```
sudo dnf install -y java-17-openjdk-devel openssh-server rsync curl tar
sudo systemctl enable --now sshd
java -version
```

El comando `java -version` debe mostrar Java 17, porque esa es la versión usada en la guía del cluster. Luego se crea el mismo usuario utilizado por Hadoop en los demás nodos. En Ubuntu se puede hacer con:

Paso 8: Crear usuario hadup2 en Ubuntu

```
sudo adduser hadup2
sudo usermod -aG sudo hadup2
```

En Rocky Linux se puede crear el grupo y el usuario de forma explícita:

Paso 9: Crear usuario hadup2 en Rocky Linux

```
sudo groupadd hadup2
sudo useradd -m -g hadup2 hadup2
sudo passwd hadup2
sudo usermod -aG wheel hadup2
```

Después se entra como el usuario `hadup2`:

Paso 10: Entrar como usuario hadup2

```
sudo su - hadup2
```

Desde el NameNode se copia la instalación de Hadoop hacia el nuevo nodo:

Paso 11: Copiar Hadoop al nuevo nodo

```
rsync -av /opt/hadoop-3.5.0 hadup2@nuevo-datanode:/tmp/
```

En el nuevo DataNode se mueve Hadoop a `/opt`, se crea el enlace simbólico y se ajustan los permisos:

Paso 12: Instalar Hadoop en el nuevo DataNode

```
sudo mv /tmp/hadoop-3.5.0 /opt/
sudo ln -sfn /opt/hadoop-3.5.0 /opt/hadoop
sudo chown -R hadup2:hadup2 /opt/hadoop-3.5.0 /opt/hadoop
```

Luego se agregan las variables de entorno al archivo `~/.bashrc` del usuario `hadup2`. En Ubuntu se puede usar:

Paso 13: Variables de entorno en Ubuntu

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=/opt/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

En Rocky Linux, la ruta de Java puede ser:

Paso 14: Variables de entorno en Rocky Linux

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk
export HADOOP_HOME=/opt/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Después se cargan las variables y se verifica Hadoop:

Paso 15: Verificar Hadoop en el nuevo nodo

```
source ~/.bashrc
hadoop version
```

El nuevo DataNode también necesita el directorio local donde HDFS guardará los bloques:

Paso 16: Crear directorio local de HDFS en el nuevo DataNode

```
sudo mkdir -p /data/hadoop/hdfs/datanode
sudo chown -R hadup2:hadup2 /data/hadoop
```

Luego, desde el NameNode, se debe permitir el acceso SSH sin contraseña hacia el nuevo nodo:

Paso 17: Copiar llave SSH al nuevo DataNode

```
ssh-copy-id hadup2@nuevo-datanode
```

Se prueba la conexión:

Paso 18: Probar SSH al nuevo nodo

```
ssh hadup2@nuevo-datanode hostname
```

Si el comando responde el hostname sin pedir contraseña, la conexión SSH quedó correctamente configurada.

Después se actualiza el archivo **workers** en el NameNode:

Paso 19: Editar workers en el NameNode

```
nano $HADOOP_HOME/etc/hadoop/workers
```

Se agrega el nuevo nodo al final del archivo:

Paso 20: Agregar nuevo nodo a workers

```
nuevo-datanode
```

Luego se copia la configuración actualizada de Hadoop al nuevo DataNode:

Paso 21: Copiar configuración al nuevo nodo

```
CONFIG_DIR="$HADOOP_HOME/etc/hadoop/"  
rsync -av "$CONFIG_DIR" "hadup2@nuevo-datanode:$CONFIG_DIR"
```

Una vez hecho esto, se puede iniciar el proceso de DataNode en la nueva máquina:

Paso 22: Iniciar DataNode en el nuevo nodo

```
ssh hadup2@nuevo-datanode "/opt/hadoop/bin/hdfs --daemon start datanode"
```

Si también se quiere que el nuevo nodo participe en YARN, se inicia el NodeManager:

Paso 23: Iniciar NodeManager en el nuevo nodo

```
ssh hadup2@nuevo-datanode "/opt/hadoop/bin/yarn --daemon start nodemanager"
```

Finalmente, desde el NameNode se revisa si el nuevo nodo fue reconocido por HDFS:

Paso 24: Verificar el nuevo DataNode en HDFS

```
hdfs dfsadmin -report
```

También se puede revisar desde la interfaz web del NameNode:

Paso 25: Interfaz web del NameNode

```
http://daniel-namenode:9870/
```

Si el nuevo DataNode aparece como nodo activo, significa que fue agregado correctamente al cluster. Desde ese momento, HDFS puede empezar a usarlo para almacenar bloques nuevos.

Si después de agregar el nodo se desea equilibrar mejor el uso del disco entre todos los DataNodes, se puede ejecutar el balanceador de HDFS:

Paso 26: Ejecutar balanceador de HDFS

```
hdfs balancer
```

Este comando ayuda a redistribuir bloques cuando algunos nodos tienen más uso de disco que otros. No cambia la estructura lógica de los archivos, sino que mueve bloques entre DataNodes para que el almacenamiento quede más balanceado.

Comprobación final	Resultado esperado
<code>ping -c 3 nuevo-datanode</code>	El NameNode recibe respuesta del nuevo nodo.
<code>ssh hadup2@nuevo-datanode hostname</code>	La conexión SSH funciona sin pedir contraseña.
<code>hadoop version</code>	La nueva máquina reconoce la instalación de Hadoop.
<code>hdfs dfsadmin -report</code>	El nuevo DataNode aparece como nodo activo.
<code>http://daniel-namenode:9870/</code>	El nuevo nodo se visualiza dentro de los nodos vivos del cluster.
<code>hdfs balancer</code>	HDFS puede redistribuir bloques para equilibrar el almacenamiento.

Cuadro 7: Verificaciones recomendadas después de agregar un nuevo nodo al cluster.

En resumen, para escalar el cluster se debe preparar la nueva máquina, configurarla igual que los demás DataNodes, agregarla al archivo `workers`, copiarle la configuración de Hadoop, iniciar sus servicios y verificar que aparezca activa en el reporte de HDFS. Con esto, el cluster aumenta su capacidad de almacenamiento y queda listo para seguir funcionando de forma distribuida.

3. Guía completa para crear HDFS en una red LAN con Ubuntu y Rocky Linux

Esta sección explica el procedimiento operativo para construir un sistema de ficheros distribuido real usando Apache Hadoop HDFS. Al final se podrán ejecutar comandos como `hdfs dfs -put`, `hdfs dfs -ls`, `hdfs dfs -get` y `hdfs fsck` para almacenar, listar, recuperar y verificar archivos distribuidos.

3.1. Versión, alcance y seguridad

Para esta guía se usa Apache Hadoop 3.5.0, publicada como versión estable en abril de 2026. Esta versión requiere Java 17 en los servidores del cluster. Si en el momento de ejecutar la práctica existe una versión estable más nueva, se puede usar, pero se deben revisar sus notas de compatibilidad antes de cambiar los comandos.

Advertencia de seguridad

La configuración propuesta es no segura y sirve para una LAN controlada de laboratorio. La documentación oficial de Hadoop advierte que, sin Kerberos, cualquier usuario con acceso de red al cluster puede leer datos de HDFS y ejecutar trabajos. En un entorno de producción se debe configurar autenticación, autorización, red privada, políticas de seguridad, etc.

3.2. Diseño del cluster en LAN

3.2.1. Topología utilizada

Se usará una topología de cinco máquinas, una por integrante del grupo. El computador de Daniel Ramírez actuará como nodo maestro del cluster y los computadores de los demás integrantes actuarán como nodos de almacenamiento. El cluster es mixto: cuatro equipos usan Ubuntu y el equipo de Ana Sofía usa Rocky Linux.

Integrante	Rol	Sistema	Hostname	IP en la LAN
Daniel Ramírez	NameNode, ResourceManager y cliente	Ubuntu	daniel-namenode	10.43.100.77
Guillermo Apon- te	DataNode y Node-Manager	Ubuntu	guillermo-datanode	10.43.98.211
Samuel Pico	DataNode y Node-Manager	Ubuntu	samuel-datanode	10.43.100.51
Ana Sofía Arbole- da	DataNode y Node-Manager	Rocky Linux	ana-datanode	10.43.97.157
Paula Losada	DataNode y Node-Manager	Ubuntu	paula-datanode	10.43.99.177

Para una práctica inicial basta con ejecutar los comandos cliente desde el equipo de Daniel Ramírez, porque allí estarán el NameNode y las herramientas de administración. Las IP registradas corresponden a la red LAN del laboratorio.

3.2.2. Puertos importantes

Los puertos más usados durante la práctica son:

Servicio	Puerto	Uso
NameNode RPC	9000	Operaciones HDFS desde clientes y DataNodes
NameNode Web UI	9870	Consola web de HDFS
DataNode HTTP	9864	Consulta web de cada DataNode
ResourceManager Web UI	8088	Consola web de YARN
SSH	22	Arranque remoto de daemons con scripts Hadoop

3.3. Preparación de las máquinas

3.3.1. Paso 1. Fijar hostnames

Ejecutar en: NameNode y todos los DataNodes.

Ejecutar en cada máquina el comando correspondiente:

Paso 27: Definir hostname en cada máquina

```
# En el equipo de Daniel Ramírez
sudo hostnamectl set-hostname daniel-namenode

# En el equipo de Guillermo Aponte
sudo hostnamectl set-hostname guillermo-datanode

# En el equipo de Samuel Pico
sudo hostnamectl set-hostname samuel-datanode

# En el equipo de Ana Sofía Arboleda
sudo hostnamectl set-hostname ana-datanode

# En el equipo de Paula Losada
sudo hostnamectl set-hostname paula-datanode
```

Cerrar y abrir la sesión de terminal para que el cambio se vea reflejado.

3.3.2. Paso 2. Configurar resolución de nombres

Ejecutar en: NameNode y todos los DataNodes.

En las cinco máquinas editar `/etc/hosts`:

Paso 28: Editar `/etc/hosts`

```
sudo nano /etc/hosts
```

Agregar las IP reales del laboratorio:

Paso 29: Ejemplo de `/etc/hosts` para la LAN

```
10.43.100.77 daniel-namenode
10.43.98.211 guillermo-datanode
10.43.100.51 samuel-datanode
10.43.97.157 ana-datanode
10.43.99.177 paula-datanode
```

Verificar conectividad desde cada máquina:

Paso 30: Prueba de conectividad entre nodos

```
ping -c 3 daniel-namenode
ping -c 3 guillermo-datanode
ping -c 3 samuel-datanode
ping -c 3 ana-datanode
ping -c 3 paula-datanode
```

Si alguno falla, primero corregir IP, cableado, Wi-Fi, firewall o máscara de red. Hadoop no funcionará si los nodos no se resuelven por nombre.

3.3.3. Paso 3. Instalar dependencias

Ejecutar en: NameNode y todos los DataNodes, usando el bloque de comandos correspondiente a cada sistema operativo.

Cuando haya instrucciones separadas por sistema operativo, Ana Sofía debe ejecutar primero el recuadro rojo de Rocky Linux. Después, los demás integrantes ejecutan el bloque correspondiente a Ubuntu.

Ana Sofía - Rocky Linux. Ejecutar primero en `ana-datanode`:

```
sudo dnf install -y java-17-openjdk-devel openssh-server rsync curl tar
sudo systemctl enable --now sshd
java -version
```

En los equipos con Ubuntu, ejecutar:

Paso 31: Instalar dependencias en Ubuntu

```
sudo apt update
sudo apt install -y openjdk-17-jdk openssh-server rsync curl tar
sudo systemctl enable --now ssh
java -version
```

El comando `java -version` debe mostrar Java 17. Si aparecen varias versiones de Java, seleccionar Java 17 con `update-alternatives`. El comando existe tanto en Ubuntu como en Rocky Linux:

Paso 32: Seleccionar Java 17 si hay varias versiones

```
sudo update-alternatives --config java
readlink -f $(which java)
```

3.3.4. Paso 4. Crear usuario `hadup2`

Ejecutar en: NameNode y todos los DataNodes, usando el bloque de comandos correspondiente a cada sistema operativo.

Para evitar ejecutar Hadoop como `root`, crear el mismo usuario en todas las máquinas:

Ana Sofía - Rocky Linux. Crear primero el grupo y el usuario en `ana-datanode`. No se debe asumir que `useradd -m hadup2` crea automáticamente el grupo `hadup2`; si el grupo no existe, comandos posteriores como `chown hadup2:hadup2` pueden fallar. Por eso se crea el grupo explícitamente y se usa como grupo primario del usuario. Para darle permisos administrativos en Rocky Linux, agregar `hadup2` al grupo `wheel` con `sudo usermod -aG wheel hadup2`:

```
sudo groupadd hadup2
sudo useradd -m -g hadup2 hadup2
sudo passwd hadup2
sudo usermod -aG wheel hadup2
```

En los equipos con Ubuntu, crear el usuario y darle permisos administrativos con el grupo `sudo`:

Paso 33: Crear usuario hadup2 en Ubuntu

```
sudo adduser hadup2
sudo usermod -aG sudo hadup2
```

Entrar como ese usuario para continuar la instalación:

Paso 34: Cambiar al usuario hadup2

```
sudo su - hadup2
```

Todos los pasos siguientes se ejecutan como `hadup2`, salvo los comandos que empiecen con `sudo`.

3.4. Instalación de Hadoop

3.4.1. Paso 5. Descargar Hadoop en el NameNode

Ejecutar en: solo en el NameNode.

En `daniel-namenode`, como usuario `hadup2`:

Paso 35: Descargar Apache Hadoop 3.5.0

```
cd /tmp
HADOOP_BASE="https://downloads.apache.org/hadoop/core/stable"
curl -O "$HADOOP_BASE/hadoop-3.5.0.tar.gz"
curl -O "$HADOOP_BASE/hadoop-3.5.0.tar.gz.sha512"
sha512sum -c hadoop-3.5.0.tar.gz.sha512
```

Si la verificación falla, no continuar: borrar el archivo y descargarlo de nuevo desde un espejo oficial de Apache.

3.4.2. Paso 6. Instalar Hadoop en /opt

Ejecutar en: primero en el NameNode. Las variables de entorno también se agregan después en todos los DataNodes.

En `daniel-namenode`:

Paso 36: Descomprimir e instalar Hadoop

```
sudo tar -xzf /tmp/hadoop-3.5.0.tar.gz -C /opt
sudo ln -sfn /opt/hadoop-3.5.0 /opt/hadoop
sudo chown -R hadup2:hadup2 /opt/hadoop-3.5.0 /opt/hadoop
```

Crear variables de entorno para el usuario `hadup2`:

Paso 37: Configurar variables de entorno en `/.bashrc`

```
nano ~/.bashrc
```

Ana Sofía - Rocky Linux. En `ana-datanode`, agregar primero esta variante:

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk
export HADOOP_HOME=/opt/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Agregar al final en los equipos con Ubuntu:

Paso 38: Variables de entorno de Hadoop en Ubuntu

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=/opt/hadoop
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Cargar la configuración:

Paso 39: Activar variables de entorno

```
source ~/.bashrc
hadoop version
```

3.4.3. Paso 7. Copiar Hadoop a los DataNodes

Ejecutar en: el bloque de copia se ejecuta en el NameNode; el bloque de instalación se ejecuta en todos los DataNodes.

Desde `daniel-namenode`, copiar la instalación a cada trabajador:

Paso 40: Distribuir Hadoop a los DataNodes

```
rsync -av /opt/hadoop-3.5.0 hadup2@guillermo-datanode:/tmp/
rsync -av /opt/hadoop-3.5.0 hadup2@samuel-datanode:/tmp/
rsync -av /opt/hadoop-3.5.0 hadup2@ana-datanode:/tmp/
rsync -av /opt/hadoop-3.5.0 hadup2@paula-datanode:/tmp/
```

En cada DataNode:

Paso 41: Instalar Hadoop copiado en cada DataNode

```
sudo mv /tmp/hadoop-3.5.0 /opt/
sudo ln -sfn /opt/hadoop-3.5.0 /opt/hadoop
sudo chown -R hadup2:hadup2 /opt/hadoop-3.5.0 /opt/hadoop
```

También agregar las mismas variables de entorno al `~/.bashrc` del usuario `hadup2` en cada DataNode y ejecutar:

Paso 42: Verificar Hadoop en cada DataNode

```
source ~/.bashrc
hadoop version
```

3.5. SSH sin contraseña entre nodos

3.5.1. Paso 8. Generar llave SSH en el NameNode

Ejecutar en: solo en el NameNode.

Los scripts `start-dfs.sh` y `start-yarn.sh` arrancan procesos remotos por SSH. Desde `daniel-namenode`, como `hadup2`:

Paso 43: Generar llave SSH sin frase de paso

```
ssh-keygen -t ed25519 -P "" -f ~/.ssh/id_ed25519
```

Copiar la llave pública a todos los nodos, incluido el propio NameNode:

Paso 44: Copiar llave pública a los nodos

```
ssh-copy-id hadup2@daniel-namenode
ssh-copy-id hadup2@guillermo-datanode
ssh-copy-id hadup2@samuel-datanode
ssh-copy-id hadup2@ana-datanode
ssh-copy-id hadup2@paula-datanode
```

Probar que no pida contraseña:

Paso 45: Probar SSH sin contraseña

```
ssh hadup2@daniel-namenode hostname
ssh hadup2@guillermo-datanode hostname
ssh hadup2@samuel-datanode hostname
ssh hadup2@ana-datanode hostname
ssh hadup2@paula-datanode hostname
```

3.6. Configuración de Hadoop

3.6.1. Paso 9. Crear directorios locales de HDFS

Ejecutar en: NameNode y todos los DataNodes, usando el bloque correspondiente.

En daniel-namenode:

Paso 46: Directorios del NameNode

```
sudo mkdir -p /data/hadoop/hdfs/namenode
sudo chown -R hadup2:hadup2 /data/hadoop
```

En cada DataNode:

Paso 47: Directorios de cada DataNode

```
sudo mkdir -p /data/hadoop/hdfs/datanode
sudo chown -R hadup2:hadup2 /data/hadoop
```

3.6.2. Paso 10. Configurar hadoop-env.sh

Ejecutar en: solo en el NameNode.

En daniel-namenode, editar:

Paso 48: Editar `hadoop-env.sh`

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

Agregar o ajustar estas líneas:

Paso 49: `JAVA_HOME` y usuario de daemons

```
if [ -d /usr/lib/jvm/java-17-openjdk-amd64 ]; then
    export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
elif [ -d /usr/lib/jvm/java-17-openjdk ]; then
    export JAVA_HOME=/usr/lib/jvm/java-17-openjdk
fi

export HDFS_NAMENODE_USER=hadup2
export HDFS_DATANODE_USER=hadup2
export HDFS_SECONDARYNAMENODE_USER=hadup2
export YARN_RESOURCEMANAGER_USER=hadup2
export YARN_NODEMANAGER_USER=hadup2
```

3.6.3. Paso 11. Configurar `core-site.xml`

Ejecutar en: solo en el NameNode.

Editar `$HADOOP_HOME/etc/hadoop/core-site.xml` en `daniel-namenode`:

Paso 50: `core-site.xml`

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://daniel-namenode:9000</value>
  </property>
</configuration>
```

3.6.4. Paso 12. Configurar `hdfs-site.xml`

Ejecutar en: solo en el NameNode.

Editar `$HADOOP_HOME/etc/hadoop/hdfs-site.xml`:

Paso 51: hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.heartbeat.interval</name>
    <value>1</value>
  </property>

  <property>
    <name>dfs.namenode.heartbeat.recheck-interval</name>
    <value>10000</value>
  </property>

  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///data/hadoop/hdfs/namenode</value>
  </property>

  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///data/hadoop/hdfs/datanode</value>
  </property>

  <property>
    <name>dfs.replication</name>
    <value>3</value>
  </property>

  <property>
    <name>dfs.blocksize</name>
    <value>134217728</value>
  </property>
</configuration>
```

El valor `dfs.replication=3` exige tres réplicas por bloque. Como el cluster tiene cuatro DataNodes, Hadoop puede distribuir réplicas entre varios equipos y mantener lectura disponible aunque falle temporalmente un DataNode.

3.6.5. Paso 13. Configurar workers

Ejecutar en: solo en el NameNode.

Editar `$HADOOP_HOME/etc/hadoop/workers`:

Paso 52: workers

```
guillermo-datanode
samuel-datanode
ana-datanode
paula-datanode
```

Este archivo lista los nodos donde los scripts de Hadoop arrancarán DataNodes y NodeManagers por SSH.

3.6.6. Paso 14. Configurar YARN y MapReduce

Ejecutar en: solo en el NameNode.

Aunque HDFS puede funcionar sin YARN, se incluye una configuración mínima para ejecutar trabajos MapReduce de prueba.

Editar `$HADOOP_HOME/etc/hadoop/yarn-site.xml`:

Paso 53: yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>daniel-namenode</value>
  </property>

  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>

  <property>
    <name>yarn.nodemanager.env-whitelist</name>
    <value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,
    CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_HOME,PATH,LANG,TZ,
    HADOOP_MAPRED_HOME</value>
  </property>
</configuration>
```

Editar `$HADOOP_HOME/etc/hadoop/mapred-site.xml`:

Paso 54: mapred-site.xml

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>

  <property>
    <name>mapreduce.application.classpath</name>
    <value>$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/*:$HADOOP_MAPRED_HOME/share
    /hadoop/mapreduce/lib/*</value>
  </property>

  <property>
    <name>yarn.app.mapreduce.am.env</name>
    <value>HADOOP_MAPRED_HOME=/opt/hadoop</value>
  </property>

  <property>
    <name>mapreduce.map.env</name>
    <value>HADOOP_MAPRED_HOME=/opt/hadoop</value>
  </property>

  <property>
    <name>mapreduce.reduce.env</name>
    <value>HADOOP_MAPRED_HOME=/opt/hadoop</value>
  </property>
</configuration>
```


3.6.7. Paso 15. Distribuir la configuración

Ejecutar en: solo en el NameNode.

Desde `daniel-namenode`, copiar los archivos configurados a cada DataNode:

Paso 55: Copiar configuración Hadoop a los trabajadores

```
CONFIG_DIR="$HADOOP_HOME/etc/hadoop/"
rsync -av "$CONFIG_DIR" "hadup2@guillermo-datanode:$CONFIG_DIR"
rsync -av "$CONFIG_DIR" "hadup2@samuel-datanode:$CONFIG_DIR"
rsync -av "$CONFIG_DIR" "hadup2@ana-datanode:$CONFIG_DIR"
rsync -av "$CONFIG_DIR" "hadup2@paula-datanode:$CONFIG_DIR"
```

3.7. Arranque del cluster

3.7.1. Paso 16. Formatear HDFS

Ejecutar en: solo en el NameNode.

El formateo se hace una sola vez, antes del primer arranque. Ejecutar solo en `daniel-namenode`:

Paso 56: Formatear NameNode

```
hdfs namenode -format
```

No repetir este comando en un cluster con datos

Volver a formatear el NameNode borra los metadatos de HDFS y deja inaccesibles los archivos existentes. Solo se debe repetir si se quiere reiniciar el laboratorio desde cero y se han detenido los servicios.

3.7.2. Paso 17. Iniciar HDFS

Ejecutar en: solo en el NameNode.

En `daniel-namenode`:

Paso 57: Iniciar HDFS

```
start-dfs.sh
```

Verificar procesos:

Paso 58: Verificar procesos Java con `jps`

```
jps
ssh hadup2@guillermo-datanode jps
ssh hadup2@samuel-datanode jps
ssh hadup2@ana-datanode jps
ssh hadup2@paula-datanode jps
```

Resultado esperado:

- En `daniel-namenode`: NameNode y SecondaryNameNode.
- En `guillermo-datanode`, `samuel-datanode`, `ana-datanode` y `paula-datanode`: DataNode.

Abrir la interfaz web de HDFS desde un navegador conectado a la LAN:

Paso 59: URL de NameNode Web UI

```
http://daniel-namenode:9870/
```

3.7.3. Paso 18. Iniciar YARN

Ejecutar en: solo en el NameNode.

En daniel-namenode:

Paso 60: Iniciar YARN

```
start-yarn.sh
```

Verificar:

Paso 61: Procesos esperados de YARN

```
jps
ssh hadup2@guillermo-datanode jps
ssh hadup2@samuel-datanode jps
ssh hadup2@ana-datanode jps
ssh hadup2@paula-datanode jps
```

Resultado esperado:

- En daniel-namenode: ResourceManager.
- En guillermo-datanode, samuel-datanode, ana-datanode y paula-datanode: NodeManager.

La consola web de YARN debe estar disponible en:

Paso 62: URL de ResourceManager Web UI

```
http://daniel-namenode:8088/
```

3.8. Pruebas de HDFS

3.8.1. Paso 19. Crear directorio de usuario en HDFS

Ejecutar en: solo en el NameNode.

En daniel-namenode:

Paso 63: Crear directorio de usuario en HDFS

```
hdfs dfs -mkdir -p /user/hadup2
hdfs dfs -ls /user
```

3.8.2. Paso 20. Subir un archivo a HDFS

Ejecutar en: solo en el NameNode.

Crear un archivo local de prueba:

Paso 64: Crear archivo local de prueba

```
echo "archivo de prueba para HDFS en LAN" > prueba.txt
```

Subirlo al sistema distribuido:

Paso 65: Subir archivo a HDFS

```
hdfs dfs -put prueba.txt /user/hadup2/  
hdfs dfs -ls /user/hadup2  
hdfs dfs -cat /user/hadup2/prueba.txt
```

3.8.3. Paso 21. Descargar y comparar

Ejecutar en: solo en el NameNode.

Descargar el archivo desde HDFS y compararlo con el original:

Paso 66: Descargar archivo desde HDFS

```
hdfs dfs -get /user/hadup2/prueba.txt prueba_descargada.txt  
cmp prueba.txt prueba_descargada.txt
```

Si `cmp` no imprime nada, los archivos son idénticos.

3.8.4. Paso 22. Verificar bloques y réplicas

Ejecutar en: solo en el NameNode.

Ejecutar:

Paso 67: Inspeccionar archivo en HDFS

```
hdfs fsck /user/hadup2/prueba.txt -files -blocks -locations
```

La salida debe mostrar los bloques del archivo y las ubicaciones en los DataNodes. Con `dfs.replication=3`, cada bloque debe tener tres réplicas mientras al menos tres DataNodes estén activos.

3.8.5. Paso 23. Probar tolerancia básica a fallos

Ejecutar en: solo en el NameNode.

Detener temporalmente un DataNode:

Cuando se ejecuta un comando por SSH, la sesión remota no siempre carga las variables de entorno de `/.bashrc`. Por eso se usa la ruta completa `/opt/hadoop/bin/hdfs` y no solamente `hdfs`; así se evita el error `hdfs: command not found`.

Paso 68: Detener un DataNode para simular fallo

```
ssh hadup2@guillermo-datanode "/opt/hadoop/bin/hdfs --daemon stop datanode"
```

Intentar leer el archivo:

Paso 69: Leer archivo con un DataNode detenido

```
hdfs dfs -cat /user/hadup2/prueba.txt
hdfs fsck /user/hadup2/prueba.txt -files -blocks -locations
```

Si existe otra réplica disponible, la lectura debe funcionar. Luego reactivar el DataNode:

Paso 70: Reactivar DataNode

```
ssh hadup2@guillermo-datanode "/opt/hadoop/bin/hdfs --daemon start datanode"
```

Esperar unos segundos y revisar la salud del cluster:

Paso 71: Revisar estado general de HDFS

```
hdfs dfsadmin -report
```

3.9. Prueba opcional con MapReduce

Para validar que YARN también funciona, ejecutar el ejemplo `grep` incluido con Hadoop:

Paso 72: Preparar entrada para MapReduce

```
hdfs dfs -mkdir -p /user/hadup2/input
hdfs dfs -put $HADOOP_HOME/etc/hadoop/*.xml /user/hadup2/input
```

Ejecutar el trabajo:

Paso 73: Ejecutar ejemplo MapReduce

```
EXAMPLES_JAR="$HADOOP_HOME/share/hadoop/mapreduce"
EXAMPLES_JAR="$EXAMPLES_JAR/hadoop-mapreduce-examples-3.5.0.jar"

hadoop jar "$EXAMPLES_JAR" \
grep \
/user/hadup2/input \
/user/hadup2/output \
'dfs[a-z.]+'
```

Consultar resultado:

Paso 74: Ver salida del trabajo MapReduce

```
hdfs dfs -cat /user/hadup2/output/*
```

Si se repite la prueba, borrar antes el directorio de salida:

Paso 75: Borrar salida previa

```
hdfs dfs -rm -r /user/hadup2/output
```

3.10. Apagado del cluster

Detener servicios desde `daniel-namenode`:

Paso 76: Apagar YARN y HDFS

```
stop-yarn.sh  
stop-dfs.sh
```

Confirmar que no queden procesos Hadoop:

Paso 77: Verificar apagado

```
jps  
ssh hadup2@guillermo-datanode jps  
ssh hadup2@samuel-datanode jps  
ssh hadup2@ana-datanode jps  
ssh hadup2@paula-datanode jps
```

3.11. Problemas encontrados durante la práctica

3.11.1. El DataNode no aparece en la web del NameNode

Tuvimos que revisar que `guillermo-datanode`, `samuel-datanode`, `ana-datanode` y `paula-datanode` estén en `workers`, que `/etc/hosts` resuelva los nombres correctamente y que el puerto SSH funcione. También se pueden revisar logs en:

Paso 78: Logs de Hadoop

```
ls $HADOOP_HOME/logs  
tail -n 80 $HADOOP_HOME/logs/hadoop-hadup2-datanode-*.log
```

3.11.2. SSH sigue pidiendo contraseña

Verificar permisos:

Paso 79: Permisos correctos de SSH

```
chmod 700 ~/.ssh  
chmod 600 ~/.ssh/authorized_keys  
chmod 600 ~/.ssh/id_ed25519
```

También confirmar que se está usando el usuario `hadup2` en todos los nodos.

3.11.3. Error de JAVA_HOME

Revisar la ruta real de Java:

Paso 80: Encontrar ruta de Java

```
readlink -f $(which java)
```

La ruta de `JAVA_HOME` debe apuntar al directorio base del JDK, no al binario `java`. En Ubuntu normalmente será `/usr/lib/jvm/java-17-openjdk-amd64`; en Rocky Linux suele ser `/usr/lib/jvm/java-17-openjdk`.

4. Conclusiones

La práctica permitió construir y validar un cluster Hadoop funcional en una red LAN, usando cinco máquinas heterogéneas. Con esto se comprobó que HDFS es una alternativa útil cuando se necesita centralizar el acceso a archivos distribuidos entre varios equipos y aprovechar el almacenamiento disponible en diferentes nodos del laboratorio.

También se concluye que la heterogeneidad (distribución de Linux) cambia el proceso de instalación y administración. El caso de Rocky Linux obligó a diferenciar comandos, permisos, servicios y rutas frente a Ubuntu, lo que demuestra que en un sistema distribuido no basta con que las máquinas estén conectadas; también se debe controlar la compatibilidad entre entornos.

Por último, incluir YARN amplió el alcance de la guía, porque permitió pasar de una instalación centrada solo en almacenamiento a una configuración capaz de ejecutar cargas distribuidas. Esto deja el cluster preparado no solo para guardar archivos, sino también para realizar pruebas de procesamiento sobre los datos almacenados.

5. Referencias

- Apache Hadoop. *Apache Hadoop 3.5.0 Overview*. Disponible en: <https://hadoop.apache.org/docs/r3.5.0/>
- Apache Hadoop. *Hadoop Cluster Setup*. Disponible en: <https://hadoop.apache.org/docs/r3.5.0/hadoop-project-dist/hadoop-common/ClusterSetup.html>
- Apache Hadoop. *Setting up a Single Node Cluster*. Disponible en: <https://hadoop.apache.org/docs/r3.5.0/hadoop-project-dist/hadoop-common/SingleCluster.html>
- Apache Hadoop. *Download Apache Hadoop*. Disponible en: <https://hadoop.apache.org/releases.html>